GPUMODE Lecture Study Notes: Lecture 39 Torchtitan

Core Problem the Lecture Addresses

This lecture studies **TorchTitan**, a compact PyTorch reference architecture for large-scale language model training. The central problem is not how to write a single CUDA kernel, but how to organize a complete distributed training stack that can scale from a single node to hundreds or thousands of GPUs while remaining understandable, hackable, and close to standard PyTorch.

The motivating question is:

Given a very large dataset and a transformer model, how do we train efficiently across many GPUs?

The lecture frames TorchTitan as a response to several practical bottlenecks:

- Large models cannot always fit on one GPU.
- Long sequence lengths create large activation memory pressure.
- Large datasets, possibly trillions of tokens, make single-GPU or small-node training too slow.
- Distributed training code is often difficult to read because model code, communication code, checkpointing code, and runtime configuration become deeply entangled.
- Existing frameworks can be powerful but may be difficult to modify for research experiments.

TorchTitan is presented as a **reference repo**, not a traditional framework. The intended workflow is:

$$\text{clone} \rightarrow \text{copy} \rightarrow \text{modify} \rightarrow \text{train your own model}$$

The lecture repeatedly emphasizes that TorchTitan is deliberately small, mostly Python, and intended to be read like a book. The codebase is short enough that

an engineer can inspect the training script, the parallelization logic, and the utilities in a single focused session.

> TorchTitan is not mainly about hiding distributed training behind a black-box abstraction. It is about showing how modern PyTorch distributed primitives compose into a practical large-scale training recipe.

The lecture uses TorchTitan to explain:

- Device meshes as abstractions over distributed hardware.
- Data parallelism, fully sharded data parallelism, tensor parallelism, context parallelism, and pipeline parallelism.
- Why pipeline parallelism complicates codebases.
- How distributed dataloading must respect data-parallel and non-data-parallel ranks.
- How model code can remain mostly non-intrusive while distributed strategies are applied externally.
- How activation checkpointing, compilation, FP8, distributed checkpointing, and profiling fit into a training loop.

Required Background

Distributed Training Vocabulary

A distributed training job consists of many processes. In common GPU training setups, each process controls one GPU. The total number of participating processes is called the **world size**:

$$W = \text{number of distributed ranks}$$

Each process has a unique rank:

$$r \in \{0, 1, \ldots, W - 1\}$$

The basic distributed setup usually initializes a process group:

`torch.distributed.init_process_group`

For NVIDIA GPU clusters, the communication backend is usually NCCL. NCCL provides collective communication operations such as:

- **all-reduce**: combine tensors across ranks and return the result to all ranks.

- **all-gather**: gather shards from ranks so each rank obtains a full tensor.
- **reduce-scatter**: reduce tensors and scatter shards of the result.
- **all-to-all**: exchange different tensor chunks between ranks.

These collectives are the fundamental building blocks underneath DDP, FSDP, tensor parallelism, context parallelism, and distributed checkpointing.

Data Parallelism

In ordinary data parallelism, every GPU owns a full copy of the model. Each rank receives a different mini-batch. During backward propagation, gradients are synchronized across ranks.

If each rank has model parameters θ, local loss L_r, and gradient $g_r = \nabla_\theta L_r$, then data parallelism computes:

$$g = \frac{1}{W} \sum_{r=0}^{W-1} g_r$$

The update is then:

$$\theta_{t+1} = \theta_t - \eta g$$

where η is the learning rate.

The advantage is conceptual simplicity. The disadvantage is memory duplication:

$$\text{parameter memory per GPU} = |\theta|$$

where $|\theta|$ denotes the total memory footprint of the model parameters.

Fully Sharded Data Parallelism

Fully Sharded Data Parallelism, or FSDP, shards model state across ranks. Instead of every GPU storing the full model, each rank stores only a shard of the parameters, gradients, and optimizer state.

A simplified memory model is:

$$\text{parameter memory per GPU} \approx \frac{|\theta|}{W}$$

For optimizer states such as Adam or AdamW, the savings are even more important because Adam typically stores first and second moments:

$$m_t, \quad v_t$$

A simplified full-state memory estimate for Adam in FP32 is:

$$\text{model state memory} \approx \text{parameters} + \text{gradients} + \text{first moment} + \text{second moment}$$

Thus, without sharding, the optimizer state can dominate memory. With FSDP, those states are partitioned across ranks.

The trade-off is communication. Before computing a layer, the needed parameter shards must be gathered. After backward propagation, gradients must be reduced and scattered.

Tensor Parallelism

Tensor parallelism shards individual matrix multiplications across devices. For a linear layer:

$$Y = XW$$

with

$$X \in \mathbb{R}^{B \times D_{\text{in}}}, \quad W \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}, \quad Y \in \mathbb{R}^{B \times D_{\text{out}}},$$

one may shard W across its output dimension:

$$W = \begin{bmatrix} W_0 & W_1 & \cdots & W_{p-1} \end{bmatrix}$$

Then each tensor-parallel rank computes:

$$Y_i = XW_i$$

and the full output is the concatenation:

$$Y = \begin{bmatrix} Y_0 & Y_1 & \cdots & Y_{p-1} \end{bmatrix}$$

Tensor parallelism reduces per-device compute and memory for large layers, but it introduces collectives around the layer boundaries.

Pipeline Parallelism

Pipeline parallelism partitions the model by layers. For a transformer with L layers, one can split layers into pipeline stages:

$$\{1, \ldots, L\} = S_0 \cup S_1 \cup \cdots \cup S_{p-1}$$

Each stage runs on a different group of GPUs. Microbatches are streamed through stages like an assembly line.

Pipeline parallelism is powerful but complicated because it changes the structure of the training loop. The forward and backward passes must be scheduled carefully to avoid pipeline bubbles.

A pipeline bubble is idle time caused by incomplete pipeline utilization:

$$\text{bubble fraction} \approx \frac{p-1}{m+p-1}$$

where p is the number of pipeline stages and m is the number of microbatches. More microbatches reduce bubbles but increase activation and scheduling complexity.

The lecture deliberately avoids deeply covering pipeline parallelism because it tends to force special cases throughout the codebase.

Context Parallelism

Context parallelism shards the sequence dimension. For an input tensor:

$$X \in \mathbb{R}^{B \times S \times D},$$

where B is batch size, S is sequence length, and D is hidden dimension, context parallelism partitions:

$$S = S_0 + S_1 + \cdots + S_{p-1}$$

Each rank owns a slice of the sequence. This is useful when sequence length is too large for one GPU.

The main difficulty is attention. In self-attention, each query position may attend to all key-value positions:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

If K and V are sharded across sequence dimension, attention requires a strategy to communicate or rotate key-value blocks. This connects to blockwise attention and ring attention.

Main Concepts

TorchTitan as a Reference Architecture

TorchTitan is described as a small PyTorch repository for large-scale training. It is not meant to support every model, dataset, optimizer, or modality. Instead, it demonstrates a clean pattern for assembling distributed training techniques.

The philosophy is:

- Keep the code short enough to read.
- Keep the model implementation as ordinary PyTorch.
- Apply parallelism externally through PyTorch distributed APIs.
- Encourage users to copy and modify the repository.
- Prefer composability over a huge monolithic framework.

This style is compared to projects like GPT-Fast, where the code is meant to be understandable and plagiarized for custom work.

Configuration-Driven Training

TorchTitan training is launched with a command that points to a configuration file. The configuration file controls:

- Model size and architecture.
- Dataset selection.
- Tokenizer path.
- Data parallel degree.
- Tensor parallel degree.
- Pipeline parallel degree.
- Context parallel degree.
- Activation checkpointing mode.
- Mixed precision and FP8 settings.
- Checkpoint frequency.
- Logging backend.
- Profiling settings.

The conceptual form is:

```
torchrun --nproc_per_node=8 train.py --job.config_file configs/train.toml
```

The important point is that the user experience is similar to other training repos: choose a config, launch a distributed job, log metrics, and iterate.

Train Script as the Main Spine

The main training script acts as the spine of the system. Its high-level responsibilities are:

1. Initialize logging and debug tools.
2. Set determinism and random seeds.
3. Initialize distributed process groups.
4. Build the device mesh.
5. Build tokenizer and dataloader.
6. Instantiate the model on the meta device.
7. Apply parallelization strategies.
8. Apply activation checkpointing and compilation.
9. Configure optimizer and learning-rate schedule.
10. Run the training loop.
11. Log performance and memory metrics.
12. Save and load distributed checkpoints.

A simplified skeleton is:

```
def main(job_config):
    init_logger(job_config)
    set_determinism(job_config.training.seed)

    parallel_dims = ParallelDims(
        dp_replicate=job_config.parallelism.dp_replicate,
        dp_shard=job_config.parallelism.dp_shard,
        tp=job_config.parallelism.tp,
        pp=job_config.parallelism.pp,
        cp=job_config.parallelism.cp,
    )

    world_mesh = parallel_dims.build_mesh(device_type="cuda")

    tokenizer = build_tokenizer(job_config)
    dataloader = build_dataloader(
```

```
        job_config=job_config,
        dp_rank=parallel_dims.dp_rank,
        dp_degree=parallel_dims.dp_degree,
    )

    with torch.device("meta"):
        model = build_model(job_config)

    parallelize_model(model, world_mesh, job_config)

    optimizer = build_optimizer(model, job_config)
    train(model, optimizer, dataloader, job_config)
```

This is not exact lecture code, but it captures the logic discussed in the walk-through.

Device Mesh

The **device mesh** is one of the most important abstractions in the lecture. It represents the physical GPU ranks as a logical multidimensional grid.

For example, suppose a job has:

$$W = 64$$

GPUs. We might organize them as:

$$W = d_{\text{dp}} \cdot d_{\text{tp}} \cdot d_{\text{pp}} \cdot d_{\text{cp}}$$

where:

- d_{dp} is the data-parallel dimension.
- d_{tp} is the tensor-parallel dimension.
- d_{pp} is the pipeline-parallel dimension.
- d_{cp} is the context-parallel dimension.

A mesh shape could be:

$$(d_{\text{dp}}, d_{\text{tp}}, d_{\text{pp}}, d_{\text{cp}}) = (8, 4, 2, 1)$$

because:

$$8 \cdot 4 \cdot 2 \cdot 1 = 64$$

The mesh lets PyTorch construct process groups corresponding to each logical dimension. For example, all ranks that share the same non-tensor dimensions may form a tensor-parallel group.

Visual Concept

Draw 64 GPUs arranged as a multidimensional cube. Label one axis as data parallel, one as tensor parallel, one as pipeline parallel, and one as context parallel. Show that collectives happen along a specific axis rather than across all GPUs.

ParallelDims as Configuration

TorchTitan uses a helper object similar to `ParallelDims` to store the degrees of each parallelism. Conceptually:

```
@dataclass
class ParallelDims:
    dp_replicate: int
    dp_shard: int
    tp: int
    pp: int
    cp: int

    @property
    def dp_degree(self):
        return self.dp_replicate * self.dp_shard

    @property
    def tp_enabled(self):
        return self.tp > 1

    @property
    def pp_enabled(self):
        return self.pp > 1

    @property
    def cp_enabled(self):
        return self.cp > 1
```

The key idea is that most distributed choices become simple predicates:

$$d_{\text{tp}} > 1 \quad \Rightarrow \quad \text{tensor parallelism is enabled}$$

$$d_{\text{cp}} > 1 \quad \Rightarrow \quad \text{context parallelism is enabled}$$

$$d_{\text{pp}} > 1 \quad \Rightarrow \quad \text{pipeline parallelism is enabled}$$

Non-Intrusive Model Parallelism

A major theme is that TorchTitan tries to keep model code ordinary. The model is written as a normal PyTorch transformer. Distributed strategies are then applied on top.

The conceptual pattern is:

plain model → apply tensor parallel → apply activation checkpointing → apply compile → apply FSDP

The model itself should not need to contain explicit NCCL calls, rank checks, or hand-written collectives. This differs from more intrusive frameworks where model layers are rewritten as specialized parallel layers.

The lecture clarifies that this is a goal, not an absolute guarantee. Pipeline parallelism and some model-specific structures may still require minor conditional code.

Why Order of Parallelization Matters

The order of applying transformations is not arbitrary. In TorchTitan, tensor parallelism is applied before FSDP.

A simplified reason is:

TP changes the internal computation of layers

while

FSDP changes how parameters are stored and gathered

If FSDP were applied too early, it might wrap modules before tensor parallelism has had a chance to transform their internal computation. Applying FSDP last lets it shard the already-parallelized module.

The lecture emphasizes that composability requires understanding what each strategy does to:

- Parameter layout.
- Activation layout.
- Communication boundaries.

- Forward and backward graph structure.
- Runtime prefetch behavior.

Hardware-Level Explanation

From PyTorch Process to GPU Execution

In a typical TorchTitan run, each process controls one GPU. The process executes Python code, launches CUDA kernels, and participates in NCCL collectives.

The stack is:

Python training script → PyTorch operators → CUDA kernels and NCCL collectives → GPU SM execution

Within each GPU, kernels execute across Streaming Multiprocessors, or SMs. Each SM schedules warps. A warp contains 32 threads on NVIDIA GPUs.

The distributed training code controls **which tensors live on which GPU** and **which collectives move data between GPUs**. The lower-level CUDA kernels then execute matrix multiplications, attention, normalization, optimizer updates, and communication kernels.

Memory Hierarchy

The relevant memory hierarchy is:

registers → shared memory and L1 → L2 → HBM → interconnect to other GPUs → host CPU memory → dis

Distributed training stresses more than one part of this hierarchy:

- Matrix multiplications stress tensor cores and HBM bandwidth.
- Attention stresses activation memory and softmax-related temporary storage.
- FSDP stresses inter-GPU bandwidth through all-gather and reduce-scatter.
- Checkpointing stresses GPU-to-CPU and CPU-to-storage transfer paths.
- Dataloading stresses CPU preprocessing, tokenization, storage throughput, and network filesystems.

FSDP Communication Pattern

For each wrapped module, FSDP usually performs an all-gather before forward computation so each rank can reconstruct the needed full parameter for that module.

For a module parameter tensor θ sharded across p ranks:

$$\theta = \text{concat}\left(\theta_0, \theta_1, \ldots, \theta_{p-1}\right)$$

Each rank initially stores only θ_r. Before the module runs, it participates in:

$$\theta_{\text{full}} = \text{all\_gather}\left(\theta_r\right)$$

After backward computation, gradients may be reduced and scattered:

$$\nabla\theta_r = \text{reduce\_scatter}\left(\nabla\theta_{\text{full}}\right)$$

The hardware implication is that FSDP can reduce memory dramatically, but it introduces communication that must be overlapped with computation when possible.

FSDP Granularity and Prefetching

The lecture discusses why TorchTitan wraps individual transformer blocks and then wraps the whole model.

The granularity question is:

$$\text{How large should each FSDP unit be?}$$

If the FSDP unit is too large, the model behaves closer to DDP with expensive all-gather behavior:

$$\text{large unit} \Rightarrow \text{large all-gather} \Rightarrow \text{high peak memory}$$

If the FSDP unit is too small, communication overhead becomes too frequent:

$$\text{tiny unit} \Rightarrow \text{many collectives} \Rightarrow \text{poor communication efficiency}$$

The transformer-block level is a practical balance. It allows:

- All-gather for one block while computing another block.
- Parameter memory reduction.

- Reasonable communication bucket sizes.
- Compatibility with block-level `torch.compile`.

Visual Concept

Draw three FSDP granularities side by side: whole-model wrapping, transformer-block wrapping, and per-layer wrapping. Show whole-model wrapping as one huge all-gather, transformer-block wrapping as a sequence of overlapped block fetches, and per-layer wrapping as many tiny collectives.

Hybrid Sharded Data Parallelism

The lecture briefly discusses HSDP, or Hybrid Sharded Data Parallelism. HSDP uses two levels of organization:

$$\text{outer groups replicate the model}$$

and

$$\text{inner groups shard the model}$$

Suppose there are G outer groups and each group has S GPUs:

$$W = G \cdot S$$

Within each group, FSDP shards parameters across S GPUs. Across groups, data parallelism replicates the sharded model groups.

The memory per GPU is approximately:

$$\text{parameter memory per GPU} \approx \frac{|\theta|}{S}$$

not

$$\frac{|\theta|}{W}$$

because sharding happens only within the inner group. The benefit is that all-gather rings are smaller and often stay within a node, where communication is faster.

This is useful when the model fits within a node-level FSDP group and cross-node communication would otherwise become expensive.

Tensor Parallelism Hardware Behavior

Tensor parallelism splits large GEMMs across GPUs. Consider output-column sharding:

$$Y_i = XW_i$$

Each GPU performs a smaller matrix multiplication. This reduces per-GPU compute and memory but creates communication around subsequent operations.

For example, if a later operation needs the full Y, then tensor-parallel ranks must all-gather:

$$Y = \text{all\_gather}(Y_i)$$

Alternatively, if the next operation can consume sharded layouts, communication may be delayed or reduced.

The hardware trade-off is:

$$\text{less local GEMM work} + \text{less local parameter memory} - \text{more communication}$$

Tensor parallelism is most useful when individual layers are too large or when the model needs additional parallelism beyond FSDP.

Asynchronous Tensor Parallelism

The lecture highlights asynchronous tensor parallelism as an interesting feature. Standard tensor parallelism may have blocking communication:

$$\text{compute} \rightarrow \text{communicate} \rightarrow \text{compute}$$

Asynchronous tensor parallelism tries to overlap communication and computation:

$$\text{compute chunk } 0 \parallel \text{communicate chunk } 1$$

The conceptual speedup comes from hiding communication latency under useful GEMM work.

If computation time is T_{comp} and communication time is T_{comm}, blocking execution costs approximately:

$$T_{\text{blocking}} = T_{\text{comp}} + T_{\text{comm}}$$

Perfect overlap would cost:

$$T_{\text{overlap}} = \max(T_{\text{comp}}, T_{\text{comm}})$$

The maximum possible improvement is bounded by:

$$\frac{T_{\text{blocking}}}{T_{\text{overlap}}} = \frac{T_{\text{comp}} + T_{\text{comm}}}{\max(T_{\text{comp}}, T_{\text{comm}})}$$

In practice, overlap is imperfect because of kernel scheduling, memory bandwidth contention, and collective implementation details.

Mathematical Reasoning

World Size Factorization

TorchTitan organizes the world size as a product of parallelism dimensions:

$$W = d_{\text{dp-replicate}} \cdot d_{\text{dp-shard}} \cdot d_{\text{tp}} \cdot d_{\text{pp}} \cdot d_{\text{cp}}$$

The total data-parallel degree is:

$$d_{\text{dp}} = d_{\text{dp-replicate}} \cdot d_{\text{dp-shard}}$$

A valid configuration must satisfy:

$$W = d_{\text{dp}} \cdot d_{\text{tp}} \cdot d_{\text{pp}} \cdot d_{\text{cp}}$$

This equation is the first sanity check for any distributed configuration.

Global Batch Size

If each data-parallel rank processes local batch size B_{local}, then the global batch size is:

$$B_{\text{global}} = B_{\text{local}} \cdot d_{\text{dp}}$$

If gradient accumulation is used for a steps, then:

$$B_{\text{effective}} = B_{\text{local}} \cdot d_{\text{dp}} \cdot a$$

This matters because optimization behavior depends on effective batch size, not just local GPU batch size.

Token Throughput

For language model training, a more robust throughput metric than MFU is often tokens per second:

$$\text{tokens/sec} = \frac{B_{\text{global}} \cdot S}{T_{\text{step}}}$$

where:

- B_{global} is the global batch size.
- S is sequence length.
- T_{step} is wall-clock time per optimizer step.

The lecture cautions that MFU is useful but can be ambiguous across repos, models, dtypes, and formulas. Tokens per second is often easier to compare within a fixed training setup.

Model FLOPs Utilization

Model FLOPs Utilization, or MFU, attempts to measure how much of the theoretical peak hardware compute is used for model-relevant work.

A simplified definition is:

$$\text{MFU} = \frac{\text{model FLOPs per step}}{T_{\text{step}} \cdot \text{peak FLOPs}}$$

The numerator is often estimated analytically for transformer models. A common rough approximation for dense transformer training is:

$$\text{training FLOPs per token} \approx 6N$$

where N is the number of model parameters. The factor 6 roughly accounts for forward and backward computation.

For T tokens per step:

$$\text{model FLOPs per step} \approx 6NT$$

Then:

$$\text{MFU} \approx \frac{6NT}{T_{\text{step}} \cdot F_{\text{peak}}}$$

where F_{peak} is the hardware peak FLOPs per second.

The lecture emphasizes that this is only an approximation. It becomes more nuanced when:

- Some layers use FP8 while others use BF16 or FP16.
- Attention implementations differ.
- Communication and recomputation are included or excluded differently.
- Profiling tools count operations differently.
- Different repos use different analytical formulas.

MFU Versus HFU

The lecture distinguishes MFU from Hardware FLOPs Utilization, or HFU.

HFU asks:

$$\text{HFU} = \frac{\text{actual hardware FLOPs executed per second}}{\text{peak FLOPs per second}}$$

MFU asks:

$$\text{MFU} = \frac{\text{useful model FLOPs per second}}{\text{peak FLOPs per second}}$$

If a system performs redundant computation, HFU can increase even if the model algorithm is not more efficient. MFU tries to measure useful model work, but the definition of useful work can be ambiguous.

> HFU can reward doing extra useless work quickly. MFU tries to measure useful model work, but it depends on the chosen analytical model.

FSDP Memory Model

Let:

$$P = \text{parameter memory}$$

$$G = \text{gradient memory}$$

$$O = \text{optimizer state memory}$$

For standard data parallelism:

$$M_{\text{DDP}} = P + G + O + A$$

where A is activation memory.

For FSDP over S sharding ranks:

$$M_{\text{FSDP}} \approx \frac{P + G + O}{S} + A + M_{\text{gather}}$$

where M_{gather} is transient memory needed for all-gathered parameters.

Activation checkpointing reduces A at the cost of recomputation.

Activation Checkpointing Trade-Off

Without activation checkpointing, the training step stores many intermediate activations:

$$A_{\text{stored}} = \sum_{\ell=1}^{L} A_\ell$$

where A_ℓ is activation memory for layer ℓ.

With full activation checkpointing, the memory can be reduced, but some forward operations must be recomputed during backward. The trade-off is:

$$\text{lower memory} \quad \Longleftrightarrow \quad \text{higher compute}$$

If the original step time is:

$$T_{\text{step}} = T_{\text{fwd}} + T_{\text{bwd}}$$

then checkpointing changes it to approximately:

$$T_{\text{step,ckpt}} = T_{\text{fwd}} + T_{\text{bwd}} + T_{\text{recompute}}$$

Selective activation checkpointing tries to save expensive operations and recompute cheaper ones.

Attention Memory Scaling

For standard full attention, the attention score matrix has shape:

$$S_{\text{attn}} \in \mathbb{R}^{B \times H \times S \times S}$$

where:

- B is batch size.
- H is number of attention heads.
- S is sequence length.

The memory complexity of materializing attention scores is:

$$O(BHS^2)$$

This is why long-context training becomes difficult. Context parallelism and memory-efficient attention are important because activation memory can grow quadratically with sequence length if implemented naively.

Code Walkthrough

Distributed Initialization

A simplified distributed initialization block looks like:

```
import torch
import torch.distributed as dist

def init_distributed(backend="nccl", timeout=None):
    dist.init_process_group(
        backend=backend,
        timeout=timeout,
    )

    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)

    rank = dist.get_rank()
    world_size = dist.get_world_size()

    return rank, world_size, local_rank
```

Line-by-line explanation:

- `init_process_group` creates the distributed communication context.

- `backend="nccl"` selects NVIDIA GPU collectives.
- `LOCAL_RANK` maps the current process to the GPU visible on the node.
- `torch.cuda.set_device` ensures CUDA allocations and kernels go to the correct GPU.
- `rank` identifies the process globally.
- `world_size` gives the total number of participating ranks.

Hardware behavior:

- NCCL communicators are initialized for later collectives.
- GPU memory allocations become local to the selected device.
- Later collectives will use NVLink, PCIe, InfiniBand, or another available interconnect depending on topology.

Building a Device Mesh

A representative device mesh construction looks like:

```
from torch.distributed.device_mesh import init_device_mesh

mesh = init_device_mesh(
    device_type="cuda",
    mesh_shape=(dp_degree, tp_degree, pp_degree, cp_degree),
    mesh_dim_names=("dp", "tp", "pp", "cp"),
)
```

The key abstraction is that process groups are derived from dimensions of the mesh. If tensor parallelism is along the `tp` dimension, tensor-parallel collectives only happen within that axis.

Conceptually:

$$\texttt{mesh["tp"]} \Rightarrow \text{tensor-parallel process group}$$

$$\texttt{mesh["dp"]} \Rightarrow \text{data-parallel process group}$$

This avoids manually constructing many process groups.

Data Loading Across Distributed Ranks

The lecture points out a subtle but crucial dataloading rule:

$$\text{different data-parallel ranks should receive different data}$$

but

non-data-parallel ranks for the same sample should receive the same data

This is because tensor-parallel or context-parallel ranks cooperate on the same training example. They should not tokenize different samples.

A simplified version is:

```
def build_dataloader(dataset, tokenizer, dp_rank, dp_degree, seq_len):
    sampler = DistributedSamplerLike(
        dataset=dataset,
        rank=dp_rank,
        world_size=dp_degree,
    )

    return DataLoader(
        dataset,
        sampler=sampler,
        collate_fn=lambda batch: tokenize(batch, tokenizer, seq_len),
    )
```

Here, `dp_rank` acts as the unique identity for data partitioning. Tensor-parallel ranks that share a data-parallel rank see the same examples.

This matters because if tensor-parallel ranks receive different samples, the sharded matrix multiplications become semantically invalid. They would be computing different pieces of different examples.

Checkpointable Dataloading

For trillion-token datasets, mid-epoch resumption matters. If training stops after step t, restarting from the beginning of the dataset wastes time and changes training semantics.

A checkpointable dataloader must save state such as:

dataset position, shuffle seed, worker state, rank-specific offset

A conceptual interface is:

```
state = dataloader.state_dict()
save_checkpoint({"dataloader": state})

loaded = load_checkpoint()
dataloader.load_state_dict(loaded["dataloader"])
```

This allows the job to resume from the correct location rather than the first row.

Meta Device Model Initialization

TorchTitan uses the meta device pattern to instantiate large models without immediately allocating real GPU memory.

```
with torch.device("meta"):
    model = Transformer(model_args)
```

The meta device creates tensors with shape and dtype metadata but no backing storage. This is useful because a full model may be too large to instantiate on one GPU before sharding.

The sequence is:

$$\text{create fake/meta model} \rightarrow \text{apply sharding} \rightarrow \text{materialize real shards}$$

Without this pattern, a rank may attempt to allocate the entire model before FSDP shards it, causing out-of-memory errors.

Applying Parallelism

A simplified version of the model-parallelization flow is:

```
def parallelize_model(model, mesh, job_config):
    if job_config.parallelism.tp > 1:
        apply_tensor_parallelism(
            model=model,
            mesh=mesh["tp"],
            loss_parallel=job_config.parallelism.loss_parallel,
            enable_float8=job_config.float8.enabled,
        )

    apply_activation_checkpointing(
        model=model,
        mode=job_config.activation_checkpoint.mode,
    )

    if job_config.compile.enabled:
        compile_transformer_blocks(model)

    if job_config.parallelism.dp_shard > 1:
        apply_fsdp(
            model=model,
            mesh=mesh["dp"],
            param_dtype=job_config.training.param_dtype,
            reduce_dtype=job_config.training.reduce_dtype,
        )
```

Important ordering:

1. Tensor parallelism is applied before FSDP.
2. Activation checkpointing is applied around transformer blocks.
3. Compilation is applied at block granularity.
4. FSDP is applied last because it controls storage and all-gather behavior.

Tensor Parallel Plan

A tensor-parallel plan describes how individual submodules are sharded. For a transformer block, the MLP and attention projections may be split differently.

A simplified plan might be:

```
parallelize_module(
    module=model,
    device_mesh=tp_mesh,
    parallelize_plan={
        "tok_embeddings": RowwiseParallel(),
        "layers.*.attention.wq": ColwiseParallel(),
        "layers.*.attention.wk": ColwiseParallel(),
        "layers.*.attention.wv": ColwiseParallel(),
        "layers.*.attention.wo": RowwiseParallel(),
        "layers.*.feed_forward.w1": ColwiseParallel(),
        "layers.*.feed_forward.w2": RowwiseParallel(),
        "layers.*.feed_forward.w3": ColwiseParallel(),
    },
)
```

Conceptual explanation:

- Query, key, and value projections are often column-sharded because each rank can compute a subset of heads or hidden features.
- Output projections are often row-sharded because they consume sharded activations and reduce partial results.
- MLP up-projection layers are commonly column-sharded.
- MLP down-projection layers are commonly row-sharded.

Hardware behavior:

- Each GPU performs smaller GEMMs.
- Communication is inserted where layouts need to change.
- The performance benefit depends on whether communication can be hidden or amortized.

Loss Parallelism

Loss parallelism is useful when the vocabulary dimension is large. The final logits often have shape:

$$Z \in \mathbb{R}^{B \times S \times V}$$

where V is vocabulary size. If V is large, storing and processing full logits on every rank is expensive.

Loss parallelism shards the vocabulary dimension:

$$V = V_0 + V_1 + \cdots + V_{p-1}$$

Each rank owns logits:

$$Z_i \in \mathbb{R}^{B \times S \times V_i}$$

Cross-entropy must then be computed in a distributed way. For target class y, the ordinary cross-entropy is:

$$L = -z_y + \log \sum_{j=1}^{V} e^{z_j}$$

When logits are sharded, the denominator requires a distributed sum:

$$\sum_{j=1}^{V} e^{z_j} = \sum_{i=0}^{p-1} \sum_{j \in V_i} e^{z_j}$$

This requires collectives to compute the global log-sum-exp stably.

Activation Checkpointing

TorchTitan supports multiple checkpointing styles:

- Full activation checkpointing.
- Layer-based selective activation checkpointing.
- Operator-based selective activation checkpointing.

A simplified activation checkpointing wrapper is:

```python
from torch.utils.checkpoint import checkpoint

class CheckpointedBlock(torch.nn.Module):
    def __init__(self, block):
        super().__init__()
        self.block = block

    def forward(self, x, *args, **kwargs):
        return checkpoint(self.block, x, *args, use_reentrant=False, **kwargs)
```

The idea is:

do not save all intermediates

Instead:

recompute selected intermediates during backward

Selective activation checkpointing is more refined. It tries to save expensive operations, such as large matrix multiplications, while recomputing cheaper operations.

`torch.compile` at Transformer Block Level

The lecture explains why TorchTitan applies `torch.compile` to transformer blocks rather than always compiling the whole model.

A simplified pattern is:

```python
for layer in model.layers:
    layer = torch.compile(layer)
```

The reasons are:

- FSDP communication can cause graph breaks at model boundaries.
- Transformer blocks are repeated structures and good compiler targets.
- Block-level compilation preserves FSDP prefetch behavior.
- PyTorch can reuse compilation work for repeated block structures.

A whole-model compile might appear cleaner, but if distributed wrappers create graph breaks, the compiler may see smaller fragments or lose performance.

FSDP Wrapping

A simplified FSDP application pattern is:

```python
from torch.distributed._composable.fsdp import fully_shard

def apply_fsdp(model, dp_mesh, mp_policy):
    for layer in model.layers:
        fully_shard(
            layer,
            mesh=dp_mesh,
            mp_policy=mp_policy,
        )

    fully_shard(
        model,
        mesh=dp_mesh,
        mp_policy=mp_policy,
    )
```

Interpretation:

- Each transformer block becomes an FSDP unit.
- The outer model wrapping handles parameters not inside transformer blocks, such as embeddings or output layers.
- This enables block-level all-gather and prefetch.

Hardware behavior:

- Before a block runs, parameter shards are gathered.
- While one block computes, the next block's parameters may be prefetched.
- After backward, gradients are reduced and scattered.

FP8 Training Support

The lecture discusses FP8 support for both compute and communication. FP8 is useful on newer NVIDIA architectures that support it efficiently.

A simplified configuration distinction is:

```
float8_config = {
    "enable_float8_linear": True,
    "enable_fsdp_float8_all_gather": True,
}
```

The two ideas are different:

- FP8 linear layers reduce compute and memory bandwidth for matrix multiplications.
- FP8 all-gather reduces communication volume for FSDP parameter gathering.

If a tensor has n elements, communication volume in BF16 is roughly:

$$2n \text{ bytes}$$

For FP8, it is roughly:

$$n \text{ bytes}$$

Ignoring scaling metadata, the communication volume can be approximately halved:

$$\frac{n}{2n} = \frac{1}{2}$$

This is why FP8 can improve distributed throughput even when only some operations use FP8.

Context Parallelism Context Manager

The lecture describes context parallelism as a context manager rather than intrusive model code.

A representative structure is:

```
with train_context(
    loss_parallel=loss_parallel_enabled,
    context_parallel=context_parallel_enabled,
):
    logits = model(input_ids)
    loss = loss_fn(logits, labels)
```

The context manager can activate behavior such as:

- Sequence-dimension sharding.
- Distributed attention logic.
- Loss-parallel computation.
- Composition of multiple runtime contexts in the correct order.

This avoids requiring every model forward pass to manually contain distributed logic.

Distributed Checkpointing

TorchTitan uses distributed checkpointing rather than ordinary `torch.save` for large training jobs.

A simplified checkpoint save is:

```python
import torch.distributed.checkpoint as dcp

state = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "dataloader": dataloader.state_dict(),
}

dcp.save(state, checkpoint_id=checkpoint_path)
```

The distributed checkpoint can save shards from multiple ranks instead of requiring every rank to materialize the entire model.

The lecture describes checkpointing as having stages:

$$\text{GPU} \rightarrow \text{CPU} \rightarrow \text{disk or remote storage}$$

Asynchronous checkpointing can overlap the CPU-to-disk portion with continued training.

More advanced zero-overhead checkpointing attempts to overlap even GPU-to-CPU transfer with useful computation, as long as consistency is preserved before backward modifies the relevant state.

Performance Intuition

Why TorchTitan Is Useful for Learning

TorchTitan is useful because it exposes the real pieces of distributed training without hiding them behind a huge framework. The core performance ideas are visible:

- How ranks are arranged.
- Where collectives happen.
- Why model initialization uses the meta device.
- Why FSDP wrapping granularity matters.
- Why compile is applied at transformer-block level.
- Why dataloading must be aware of data-parallel ranks.
- Why checkpointing must be distributed and resumable.

This makes it a good educational bridge between single-GPU PyTorch and large-scale training systems.

When FSDP Helps

FSDP helps when model state memory is a bottleneck. For Adam-like optimizers, sharding optimizer state can be critical.

If a model has N parameters and each FP32 state uses 4 bytes, then Adam-style states can require:

$$\text{parameter} = 4N$$

$$\text{gradient} = 4N$$

$$\text{first moment} = 4N$$

$$\text{second moment} = 4N$$

So the simplified total is:

$$16N \text{ bytes}$$

before considering activations and temporary buffers. FSDP shards this across ranks.

When FSDP Hurts

FSDP can hurt when communication dominates. If modules are too small, many small collectives occur. If modules are too large, all-gather memory spikes.

The desired regime is:

$$T_{\text{compute per block}} \geq T_{\text{communication hidden by prefetch}}$$

If this inequality is approximately true, communication can be overlapped and FSDP is efficient.

If instead:

$$T_{\text{communication}} \gg T_{\text{compute}}$$

then GPUs wait for parameters and utilization drops.

Why HSDP Can Be Better at Scale

At large world sizes, a full FSDP all-gather across all ranks can become expensive. HSDP reduces communication group size.

If full FSDP communicates across W ranks, the collective ring may span many nodes. If HSDP shards within groups of size S, then communication may stay within a node:

$$S \ll W$$

This can improve bandwidth and latency because intra-node links such as NVLink are usually faster than inter-node networks.

Why FP8 Can Help Distributed Training

FP8 helps through two pathways:

1. Faster matrix multiplication on supported hardware.
2. Lower communication volume for sharded parameter gathering.

The communication argument is especially important in FSDP-heavy training. If all-gather traffic is a bottleneck, reducing dtype size can improve step time even if not every operation is FP8.

Why MFU Must Be Interpreted Carefully

MFU is attractive because it gives one top-level number. But it can mislead because the formula depends on assumptions.

The lecture's caution can be summarized as:

MFU is useful for internal tracking, but fragile for cross-system comparison.

Tokens per second, cost per token, and stable benchmark settings are often more actionable.

Why Pipeline Parallelism Is Hard

Pipeline parallelism changes the training loop structure. Unlike FSDP or TP, which can often be applied as wrappers or layout transformations, PP requires:

- Splitting the model into stages.
- Handling missing layers on some stages.
- Scheduling microbatches.

- Managing activations crossing stage boundaries.
- Coordinating forward and backward passes across stages.
- Handling pipeline bubbles.

This explains the joke mentioned in the lecture: no codebase survives pipeline parallelism unchanged.

Common Misconceptions

Misconception: TorchTitan Is a Full Framework

TorchTitan is better understood as a reference architecture. It is intentionally small and not meant to support every possible training scenario out of the box.

The practical mindset is:

$$\text{Use TorchTitan as a starting point, not as a closed platform.}$$

Misconception: Data Parallelism Means Every Rank Gets Different Data

Only data-parallel ranks should get different data. Tensor-parallel and context-parallel ranks may need to cooperate on the same sample.

Correct rule:

$$\text{different DP rank} \Rightarrow \text{different data}$$

$$\text{same DP rank but different TP or CP rank} \Rightarrow \text{same data}$$

Misconception: FSDP Is Just DDP But Faster

FSDP is not simply faster DDP. It trades memory for communication. It is faster only when the memory savings and overlap strategy outweigh the communication overhead.

Misconception: MFU Is an Objective Universal Metric

MFU depends on formulas, dtype assumptions, model architecture, attention implementation, and what is counted as useful computation. It should not be treated as a universal absolute truth.

Misconception: FP8 Is Only About Faster Matmul

FP8 also matters for communication. In FSDP, FP8 all-gather can reduce the amount of data moved between GPUs.

Misconception: Checkpointing Is Just Saving the Model

Large-scale checkpointing includes:

- Model state.
- Optimizer state.
- Dataloader state.
- Random number generator state.
- Distributed rank-specific shards.
- Async transfer from GPU to CPU and CPU to storage.

A checkpoint that cannot resume the dataloader correctly is incomplete for long training runs.

Misconception: `torch.compile` Should Always Wrap the Whole Model

For distributed models, whole-model compilation may interact poorly with FSDP and communication boundaries. Block-level compilation can be more robust and still effective.

Practical Takeaways

Engineering Takeaways

- Start with FSDP before reaching for more complex parallelism.
- Use tensor parallelism when individual layers or compute become too large for the desired scale.
- Use context parallelism when sequence length causes activation memory pressure.
- Treat pipeline parallelism as powerful but structurally invasive.
- Think in terms of device meshes and orthogonal parallel dimensions.
- Keep the model code ordinary when possible.
- Apply distributed strategies externally through composable APIs.
- Use meta-device initialization for large models.

- Make dataloaders checkpointable for long runs.
- Prefer tokens per second and cost per token for practical performance tracking.

Debugging Takeaways

Distributed failures can be hard to debug because one hung rank may stall the entire job. TorchTitan includes debug tracing and logging utilities to help identify failures such as NCCL timeouts.

Important debug information includes:

- Rank ID.
- Local rank.
- Device assignment.
- Process group initialization.
- Memory usage.
- Collective operation progress.
- Checkpoint save and load status.
- Dataloader state.

Performance Takeaways

The most important performance question is not just:

How many GPUs are used?

but rather:

Are those GPUs doing useful work or waiting on communication?

A good distributed configuration balances:

- Local compute per GPU.
- Inter-GPU communication volume.
- Activation memory.
- Parameter and optimizer memory.
- Checkpoint overhead.
- Dataloader throughput.

Project or Experiment Ideas

Experiment 1: Single-GPU Baseline Versus FSDP

Train a small transformer on one GPU and then with FSDP across multiple GPUs. Measure:

$$\text{tokens/sec}, \quad \text{peak GPU memory}, \quad \text{step time}$$

Expected result:

- FSDP should reduce memory.
- FSDP may or may not improve speed depending on communication overhead.

Experiment 2: FSDP Wrapping Granularity

Compare three wrapping strategies:

1. Whole model.
2. Transformer block.
3. Individual sublayers.

Measure:

$$T_{\text{step}}, \quad M_{\text{peak}}, \quad \text{collective count}$$

Expected result:

- Whole-model wrapping may cause large memory spikes.
- Fine-grained wrapping may cause too many collectives.
- Transformer-block wrapping is often a practical middle ground.

Experiment 3: Activation Checkpointing Modes

Compare:

- No checkpointing.
- Full checkpointing.
- Selective checkpointing.

Measure:

$$M_{\text{activation}}, \quad T_{\text{step}}, \quad \text{tokens/sec}$$

Expected result:

- Full checkpointing saves memory but costs compute.
- Selective checkpointing should offer a better trade-off.

Experiment 4: FP8 Communication

On supported hardware, compare FSDP with and without FP8 all-gather. Measure:

$$T_{\text{all-gather}}, \quad T_{\text{step}}, \quad \text{tokens/sec}$$

Expected result:

- FP8 all-gather should reduce communication volume.
- Speedup should be more visible when communication is a bottleneck.

Experiment 5: Context Parallelism for Long Sequences

Train or benchmark attention with increasing sequence lengths:

$$S \in \{4096, 8192, 16384, 32768\}$$

Compare no context parallelism versus context parallelism. Measure:

$$M_{\text{peak}}, \quad T_{\text{attention}}, \quad \text{tokens/sec}$$

Expected result:

- Context parallelism becomes more valuable as sequence length increases.
- Communication overhead may dominate at shorter sequence lengths.

Experiment 6: Compile at Block Level

Compare:

- No compile.
- Compile transformer blocks.
- Attempt whole-model compile.

Measure:

$$T_{\text{compile}}, \quad T_{\text{step}}, \quad \text{graph breaks}$$

Expected result:

- Block-level compile should be more robust with FSDP.
- Whole-model compile may suffer from graph breaks depending on wrappers.

Final Mental Model

TorchTitan is best understood as a compact map of modern PyTorch distributed training. It starts with an ordinary transformer, places the global set of GPUs into a logical device mesh, and then applies parallel strategies along different mesh dimensions.

The mental model is:

$$\text{model too large} \Rightarrow \text{shard parameters with FSDP}$$

$$\text{layers too large} \Rightarrow \text{split matrix multiplications with TP}$$

$$\text{sequence too long} \Rightarrow \text{shard context with CP}$$

$$\text{need more scale} \Rightarrow \text{consider PP, but expect code complexity}$$

$$\text{memory too high} \Rightarrow \text{use activation checkpointing and sharded state}$$

$$\text{communication too high} \Rightarrow \text{overlap collectives, use HSDP, or reduce dtype size}$$

$$\text{training too slow} \Rightarrow \text{measure tokens/sec, memory, collectives, and compile behavior}$$

The lecture's deepest lesson is that large-scale training is not one technique. It is the composition of model layout, process groups, memory hierarchy, collective communication, checkpointing, dataloading, compiler boundaries, precision policy, and hardware topology.

> TorchTitan shows that distributed training can be made understandable when the codebase exposes the core abstractions directly: device meshes, parallel dimensions, sharded state, composable wrappers, and a plain PyTorch model at the center.